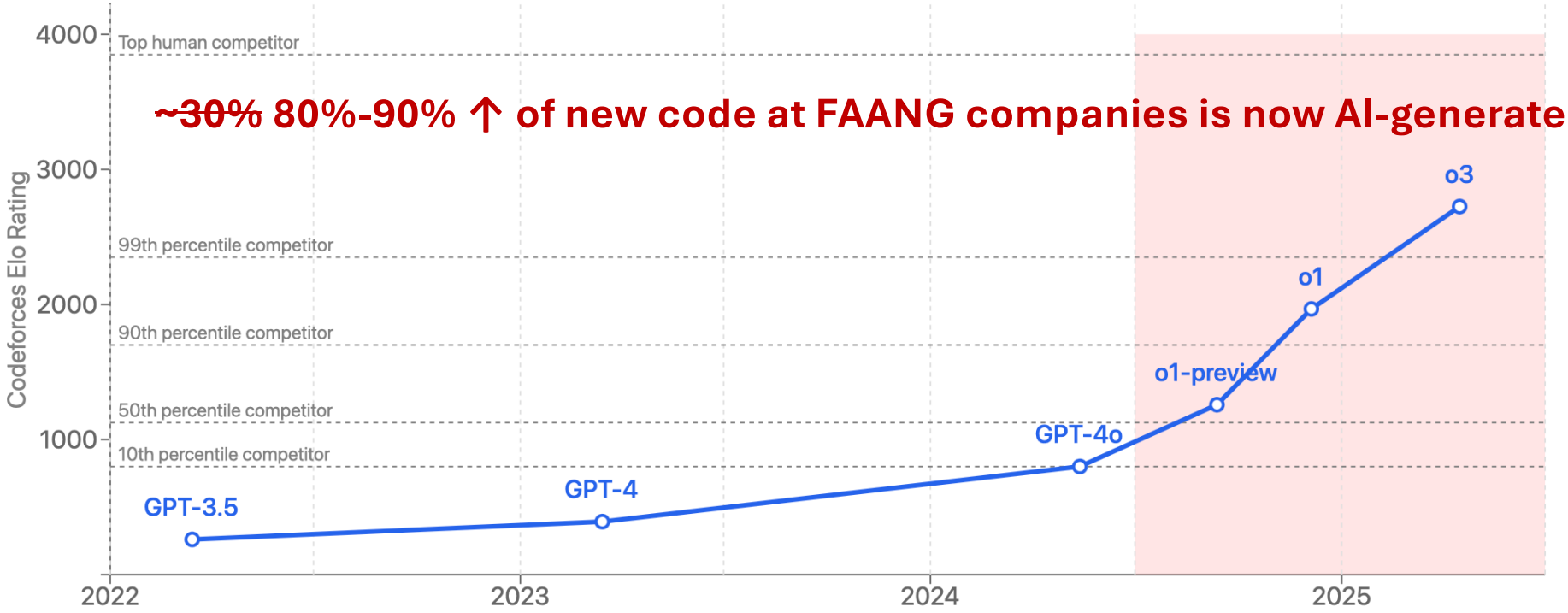
RL == Reinforcement Learning

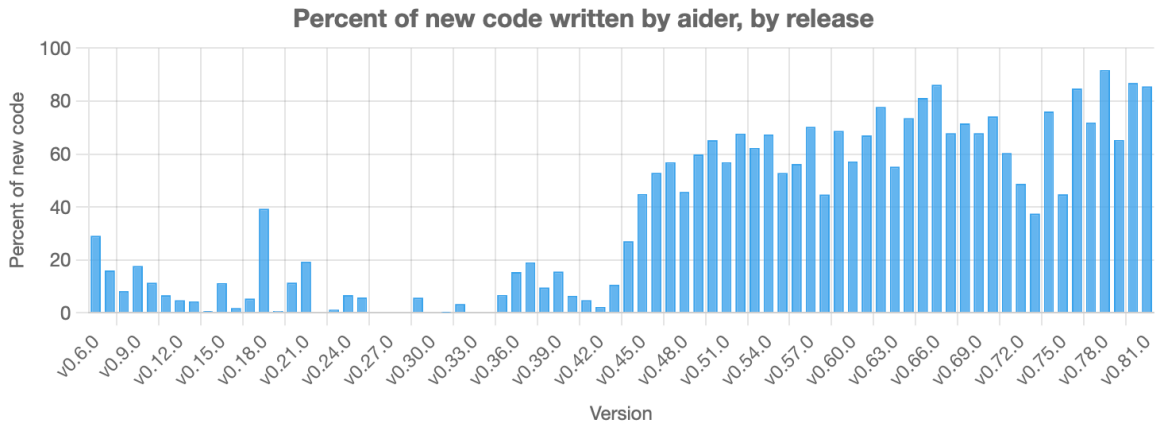
Turning point is “RL finally works”

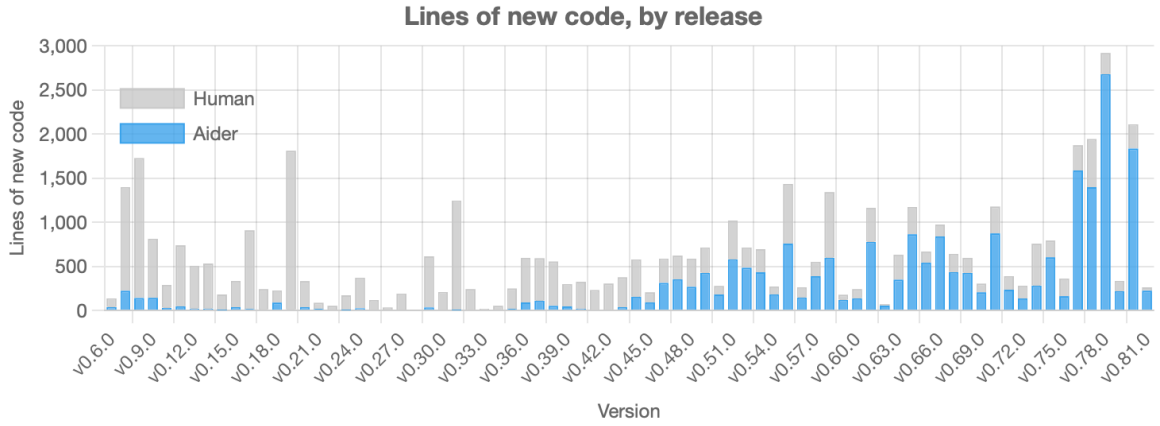


Reinforcement learning, a single recipe, now unifies software engineering, multi-step reasoning, creative writing, IMO-level math, robotics, and human-computer interaction.

Release history https://aider.chat

Aider writes most of its own code, usually about 70-80% of the new code in each release. These statistics are based on the git commit history of the aider repo.






Perhaps training for reasoning can be simpler and more sample efficient

s1: Simple test-time scaling

We show that training on only 1,000 samples with next-token prediction and controlling thinking duration via a simple test-time technique we refer to as *budget forcing* leads to a strong reasoning model that scales in performance with more test-time compute. Specifically, we construct **s1K**, Which consists of 1,000 carefully curated questions paired with reasoning traces and answers distilled from Gemini Thinking Experimental (Google, 2024).

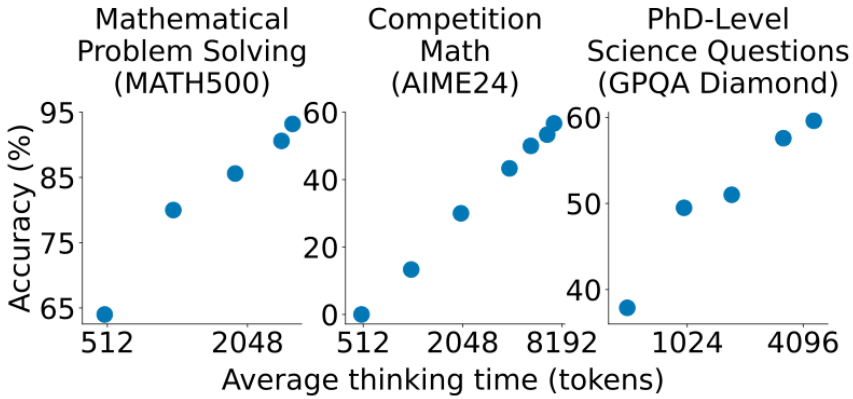


Figure 1. **Test-time scaling with s1-32B.** We benchmark **s1-32B** on reasoning-intensive tasks and vary test-time compute.

https://arxiv.org/pdf/2501.19393

s1: Simple test-time scaling

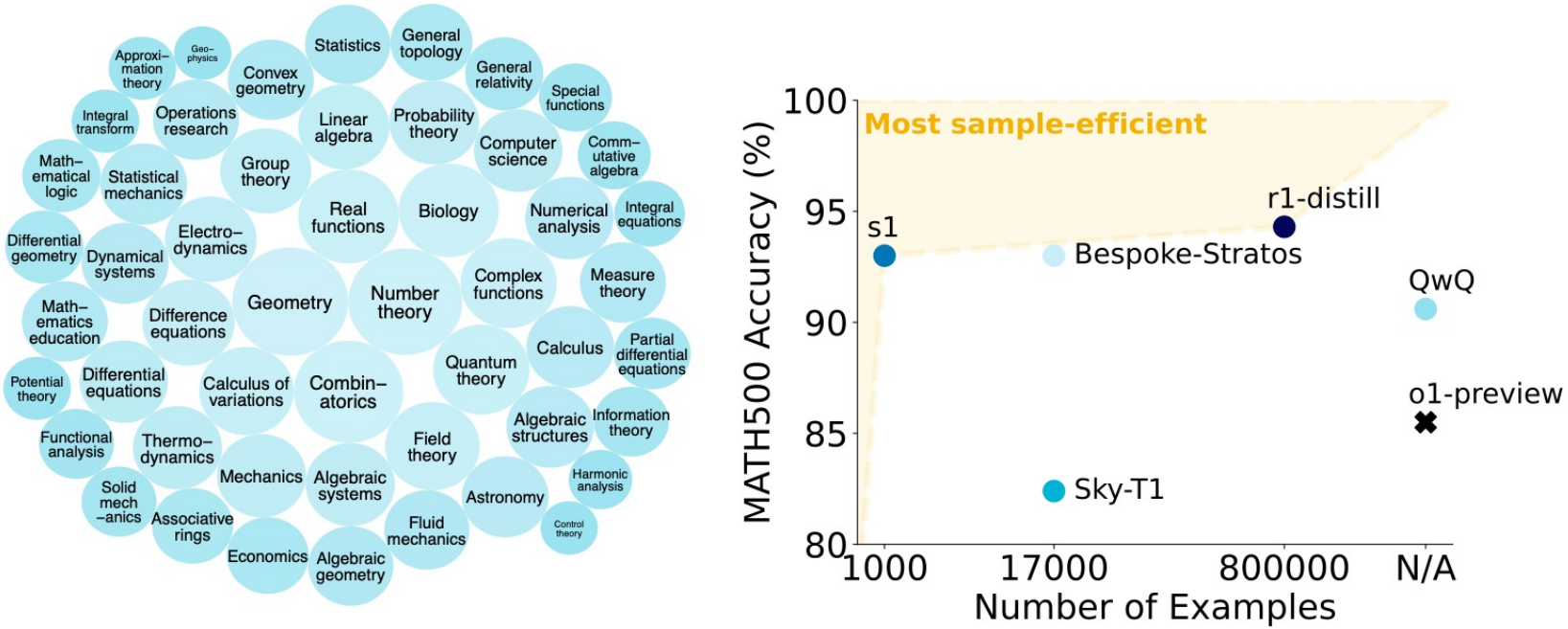
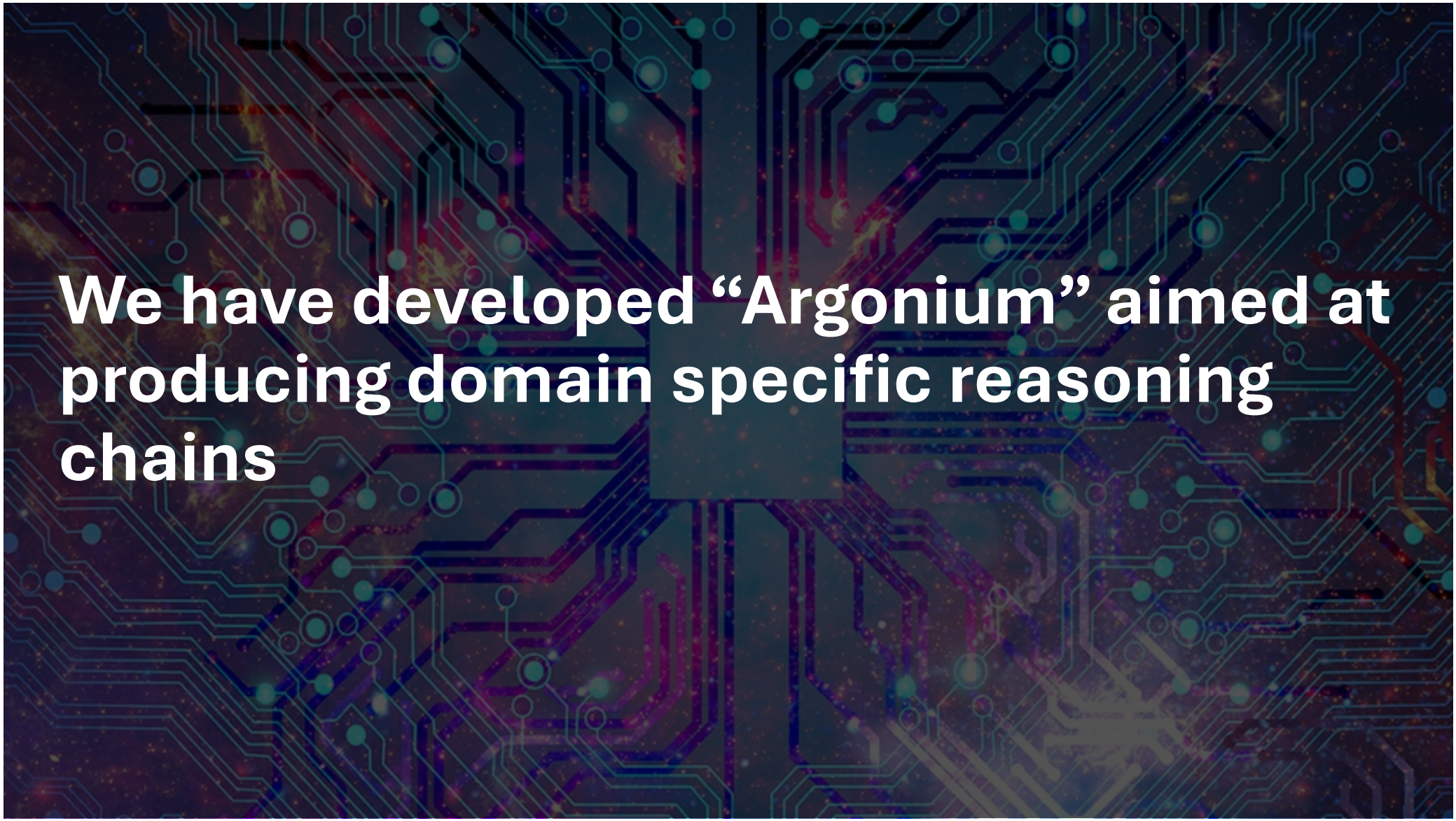
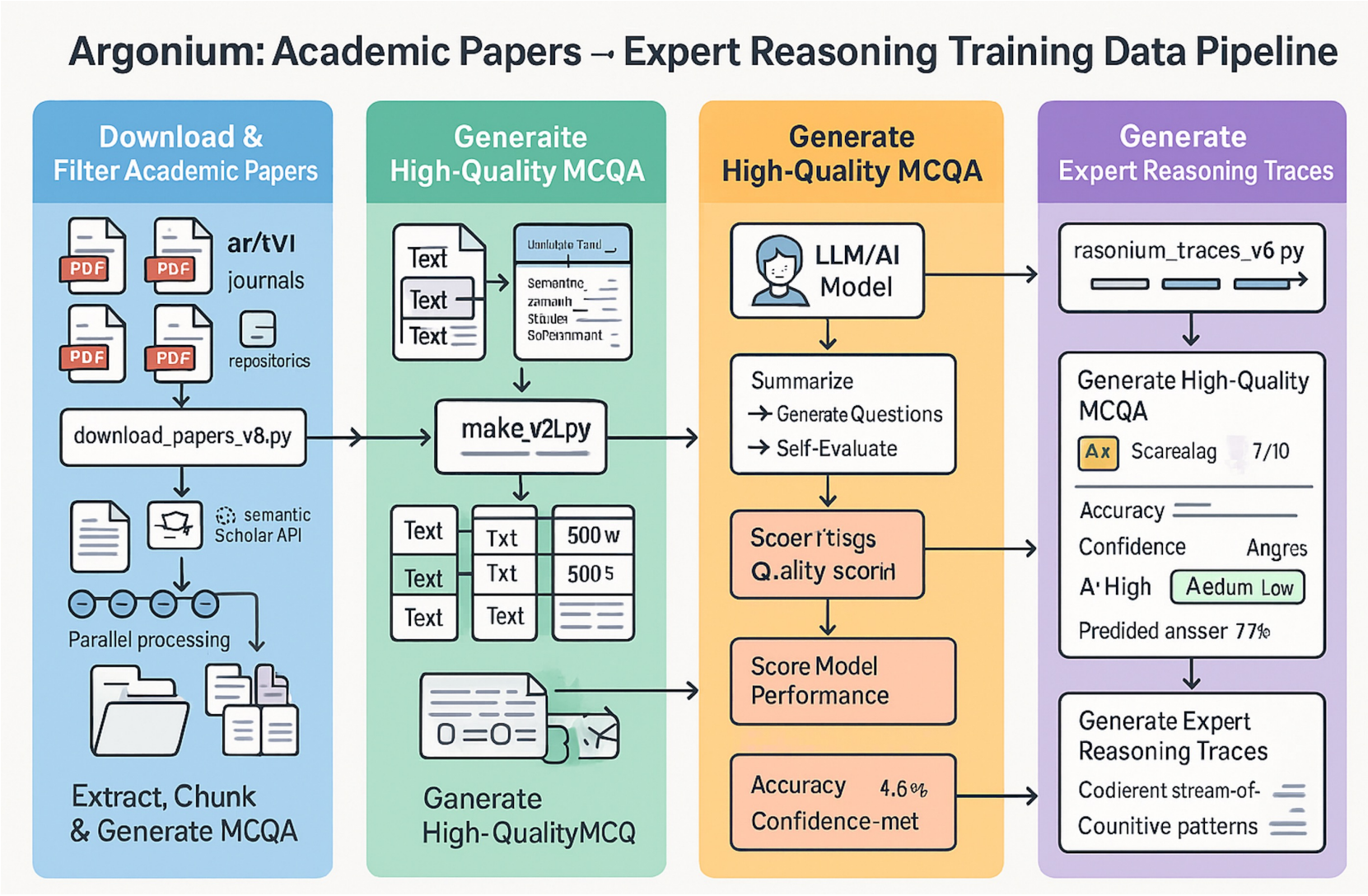


Figure 2. **s1K and s1-32B.** *(left)* **s1K** is a dataset of 1,000 high-quality, diverse, and difficult questions with reasoning traces. *(right)* **s1-32B**, a 32B parameter model finetuned on **s1K** is on the sample-efficiency frontier. See Table 1 for details on other models.

https://arxiv.org/pdf/2501.19393


We have developed “Argonium” aimed at producing domain specific reasoning chains



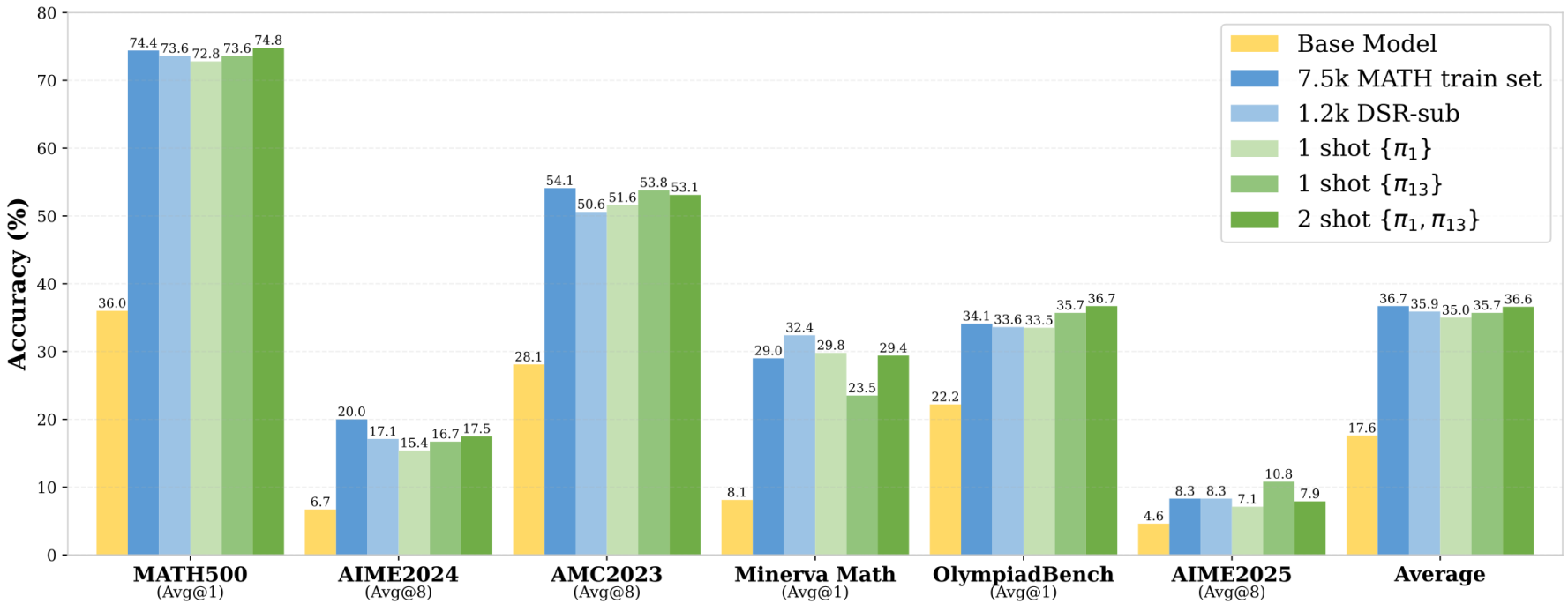
https://github.com/rick-stevens-ai/Argonium


The power of one example!

*a sequence of interactions between an agent and its environment, following a given policy

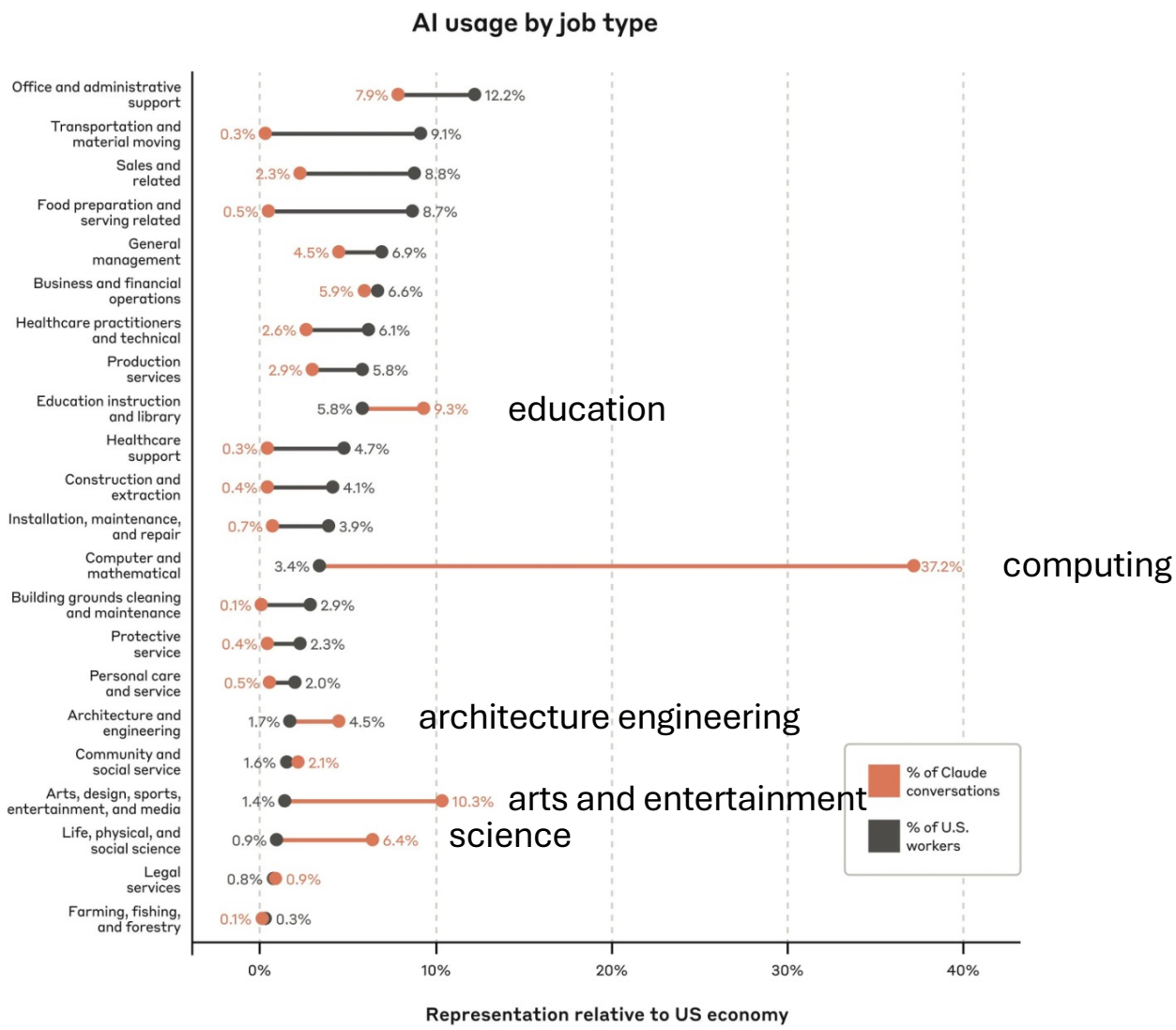
RL can renew its own training data via rollouts*

With the right incentives, a single well-chosen math problem can nearly match a 7,500-problem training set, boosting MATH500 accuracy from 36% to 73%.



Reinforcement Learning for Reasoning in Large Language Models with One Training Example (arxiv:2504.20571)


The Computing Sector adoption of AI is over 1/3 of all use of advanced AI


AI usage by job type
Office and administrative support
7.9%
12.2%
Transportation and material moving
0.3%
9.1%
Sales and related
2.3%
8.8%
Food preparation and serving related
0.5%
8.7%
General management
4.5%
6.9%
Business and financial operations
5.9%
6.6%
Healthcare practitioners and technical
2.6%
6.1%
Production services
2.9%
5.8%
Education instruction and library
5.8%
9.3%
education
Healthcare support
0.3%
4.7%
Construction and extraction
0.4%
4.1%
Installation, maintenance, and repair
0.7%
3.9%
Computer and mathematical
3.4%
37.2%
computing
Building grounds cleaning and maintenance
0.1%
2.9%
Protective service
0.4%
2.3%
Personal care and service
0.5%
2.0%
Architecture and engineering
1.7%
4.5%
architecture engineering
Community and social service
1.6%
2.1%
Arts, design, sports, entertainment, and media
1.4%
10.3%
arts and entertainment
Life, physical, and social science
0.9%
6.4%
science
Legal services
0.8%
0.9%
Farming, fishing, and forestry
0.1%
0.3%
% of Claude conversations
% of U.S. workers
0%
10%
20%
30%
40%
Representation relative to US economy

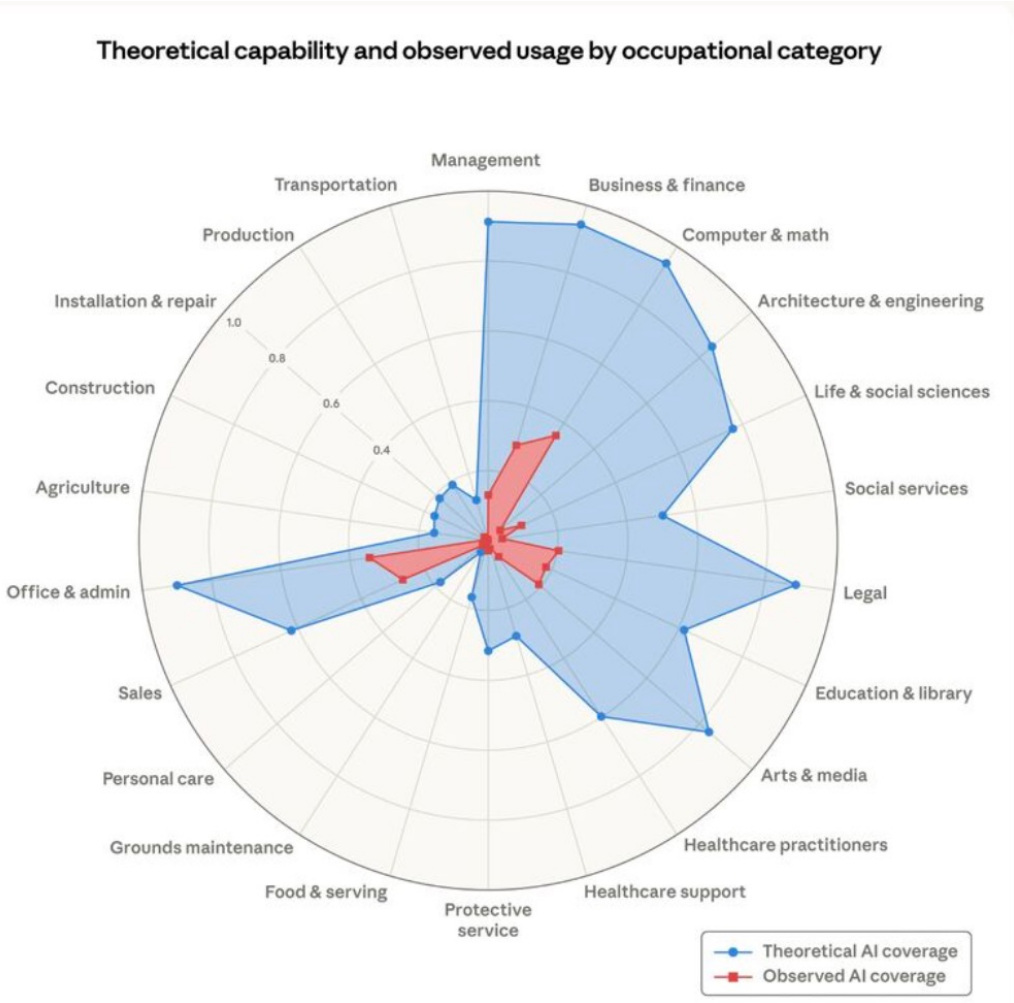


Figure 2: Theoretical capability and observed exposure by occupational category
Share of job tasks that LLMs could theoretically perform (blue area) and our own job coverage measure derived from usage data (red area).


Example productivity gains



Andrej Karpathy
@karpathy

Promote

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

Vibe coding

From Wikipedia, the free encyclopedia



This article **may contain an excessive number of citations**. Please help remove low-quality or irrelevant citations. *(June 2025)* (*Learn how and when to remove this message*)

Vibe coding (or **vibecoding**) is an approach to producing software by using artificial intelligence (AI), where a person describes a problem in a few natural language sentences as a prompt to a large language model (LLM) tuned for coding. The LLM generates software based on the description, shifting the programmer's role from manual coding to guiding, testing, and refining the AI-generated source code.[1][2][3]

Advocates of vibe coding say that it allows even amateur programmers to produce software without the extensive training and skills required for software engineering.[4] The term was introduced by Andrej Karpathy in February 2025[5][2][4][1] and listed in the Merriam-Webster Dictionary the following month as a "slang & trending" noun.[6]


Claude 3.7 Sonnet
Introducing Claude Code
Learn more
Welcome to the Claude Code research preview!
CLAUDE CODE

Scientific AI Platform Example

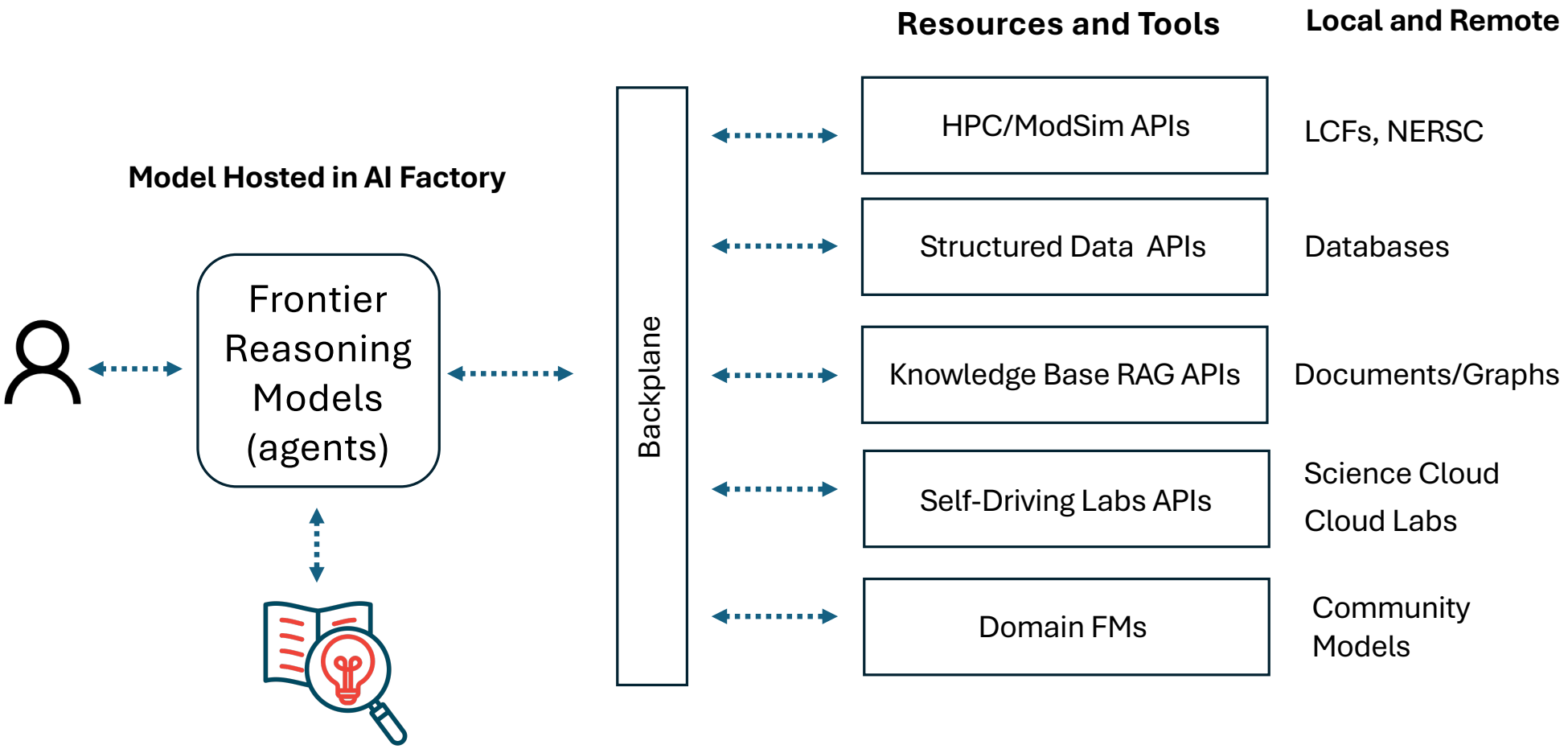


Table 3: Aggregate load, 300k context, pipelined. The 100/300 rows assume a right-sized 120B planner; 1000/5000 assume the 671B MoE (see §6.1).

	100	**300**	**1000**	**5000**
Concurrent decode streams	34	102	339	1,695
Aggregate decode	2,034 tok/s	6,102 tok/s	20,339 tok/s	101,695 tok/s
Aggregate prefill	12,203 tok/s	36,610 tok/s	122,034 tok/s	610,169 tok/s
Tokens/day	1.2 B	3.7 B	12.3 B	**61.5 B**
Tokens/year	0.4 T	1.3 T	4.5 T	**22.4 T**
Hot KV (fast memory)	234 GB	703 GB	3.6 TB	**17.9 TB**
Warm KV (all agents)	691 GB	2.1 TB	10.5 TB	**52.7 TB**
KV read bandwidth	14 TB/s	42 TB/s	214 TB/s	**1,072 TB/s**
Weight read bandwidth	7 TB/s	7 TB/s	40 TB/s	40 TB/s
KV : weight traffic	2.0 : 1	5.9 : 1	5.3 : 1	**26.6 : 1**
Sustained prefill compute	0.6 PF/s	1.9 PF/s	14.2 PF/s	70.9 PF/s
Agent-farm cores (sustained)	300	900	3,000	15,000
Sandbox memory	2.4 TB	7.2 TB	24 TB	120 TB

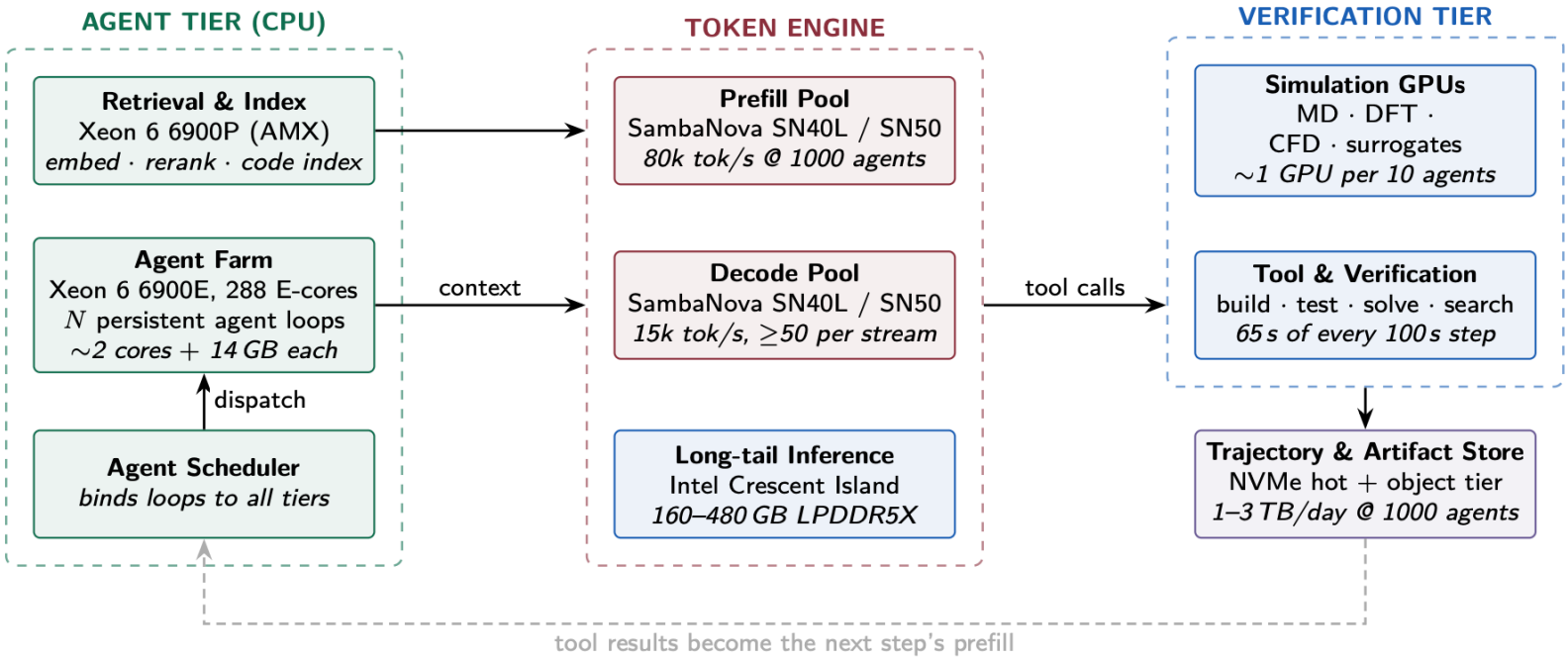
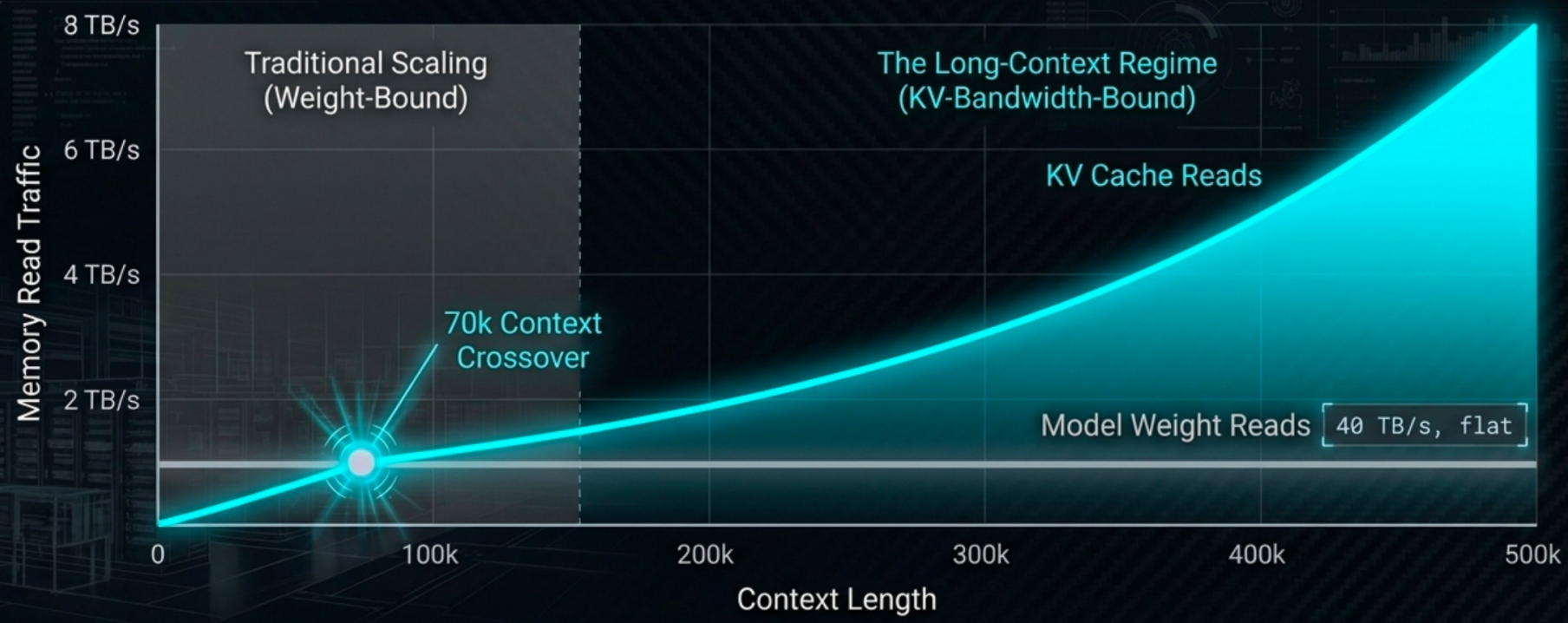


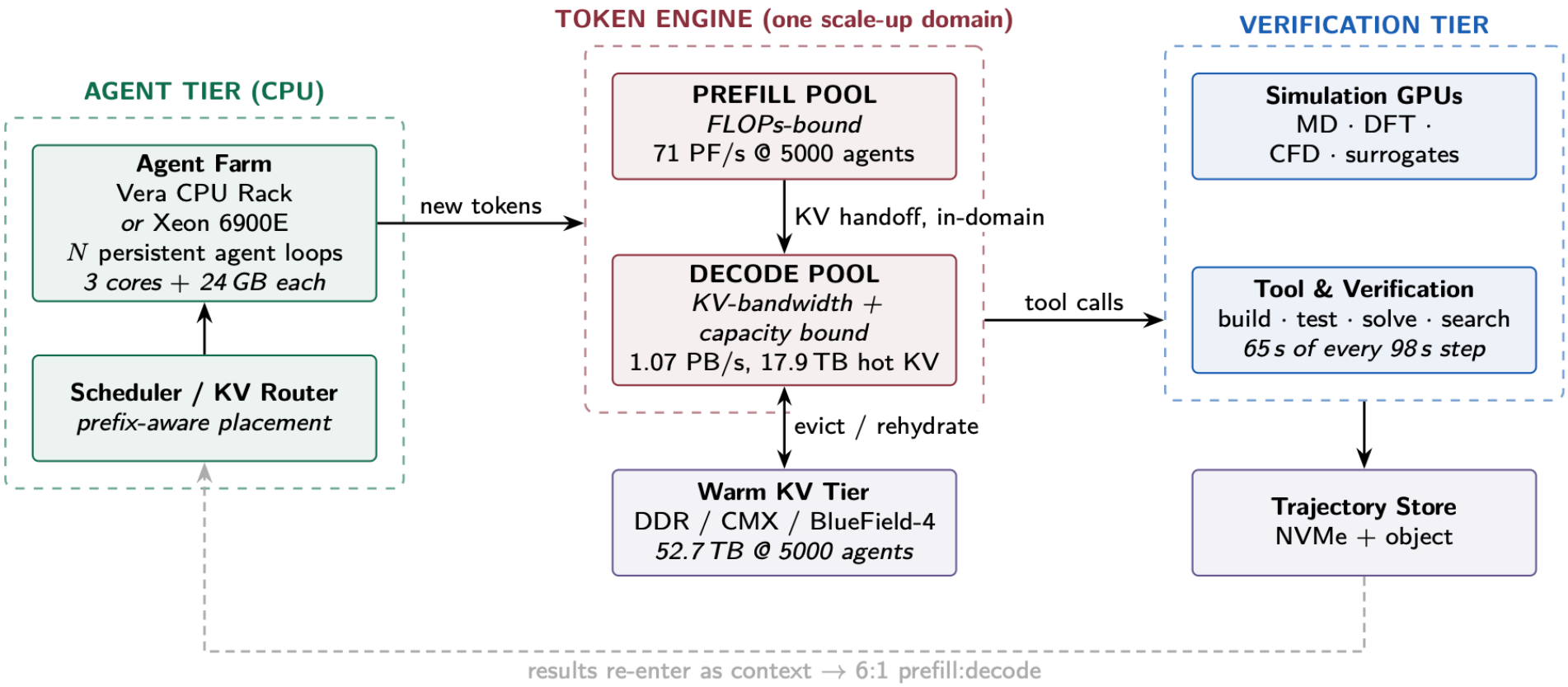
Figure 1: Tier structure of the package. The agent loop — not any single tier — is the product, which is why the scheduler that binds the tiers belongs in the package as a named deliverable rather than as an assumed integration. The dashed return path is the load driver: every tool and verification result re-enters as context on the next step, which is what makes prefill exceed decode by 5:1. Crescent Island sits in the token engine rather than the verification tier because its role is the long tail of small verifiers, draft models and per-team fine-tunes, not simulation.

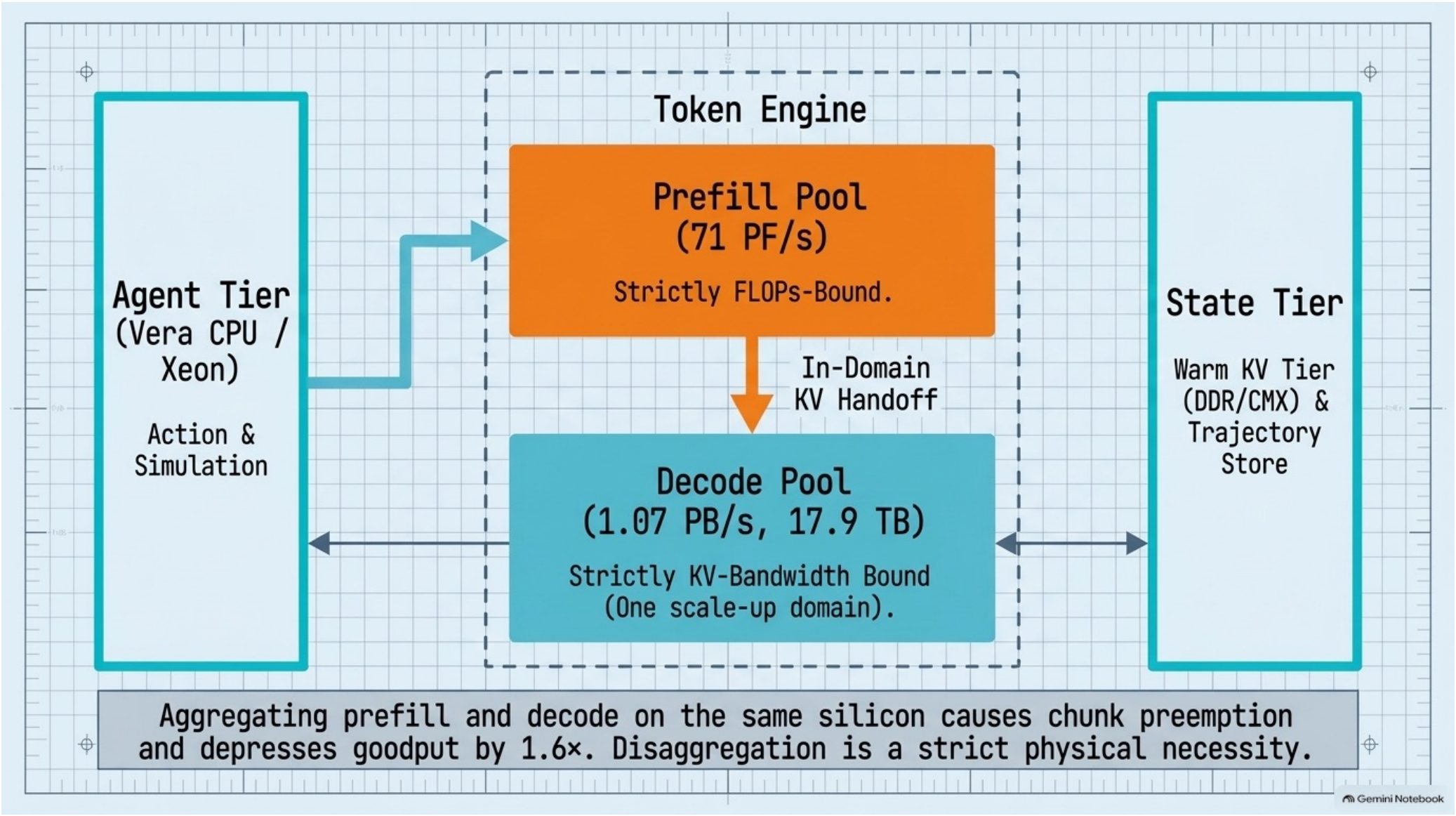
Beyond 70k Context, the Token Engine is a KV-Bandwidth Machine



At 500k context, KV traffic exceeds weight traffic by 8.9:1. Weight reads are shared across the batch, but KV reads scale violently with context multiplied by concurrent streams.

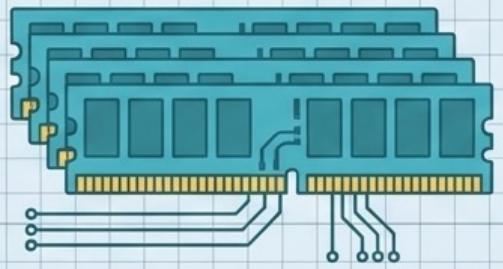
Gemini Notebook


AGENT TIER (CPU)
Agent Farm
Vera CPU Rack
or Xeon 6900E
N persistent agent loops
3 cores + 24 GB each
Scheduler / KV Router
prefix-aware placement
new tokens
TOKEN ENGINE (one scale-up domain)
PREFILL POOL
FLOPs-bound
71 PF/s @ 5000 agents
KV handoff, in-domain
DECODE POOL
KV-bandwidth + capacity bound
1.07 PB/s, 17.9 TB hot KV
evict / rehydrate
Warm KV Tier
DDR / CMX / BlueField-4
52.7 TB @ 5000 agents
tool calls
VERIFICATION TIER
Simulation GPUs
MD · DFT · CFD · surrogates
Tool & Verification
build · test · solve · search
65 s of every 98 s step
Trajectory Store
NVMe + object
results re-enter as context → 6:1 prefill:decode


Token Engine
Agent Tier
(Vera CPU / Xeon)
Action & Simulation
Prefill Pool
(71 PF/s)
Strictly FLOPs-Bound.
In-Domain
KV Handoff
Decode Pool
(1.07 PB/s, 17.9 TB)
Strictly KV-Bandwidth Bound
(One scale-up domain).
State Tier
Warm KV Tier
(DDR/CMX) &
Trajectory
Store
Aggregating prefill and decode on the same silicon causes chunk preemption and depresses goodput by 1.6×. Disaggregation is a strict physical necessity.
Gemini Notebook

KV CACHING POLICY COMPARISON

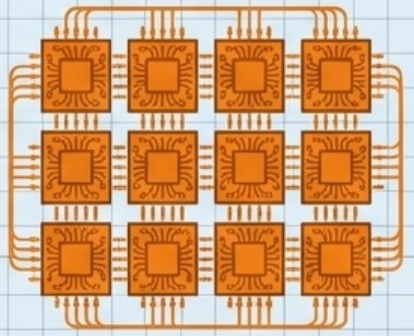
Policy A: Hold Warm KV
(Model Default)



Mechanism: Retain context between agent steps.

Physical Cost: Demands **52.7 TB** of capacity tier (DDR/CMX).

Policy B: Re-Prefill
(GPT-6 Default)



Mechanism: Recompute ~ 19% of the full context every call.

Physical Cost: Demands **302 PF/s** of extra prefill compute (approx. 75 nodes).

This is a fundamental design choice. You pay in memory watts to hold context, or compute watts to rebuild it.

Gemini Notebook

Hardware Scaling by Power & Footprint

Stack Definition	100 Agents	300 Agents	1000 Agents	5000 Agents
A0 (Intel + SambaNova, SnapStream 32k)	86 kW 5 Racks	109 kW 7 Racks	288 kW 22 Racks	1,334 kW 103 Racks
A (Intel + SN + NVIDIA)	-	-	-	4,317 kW 65 Racks
B (NVIDIA-Native)	604 kW 4 Racks	845 kW 5 Racks	1,104 kW 7 Racks	4,071 kW 23 Racks
B′ (NVIDIA + Groq LPX)	-	-	-	5,543 kW 31 Racks

SambaNova wins at small scale due to power granularity. NVIDIA NVL72 dominates at 5,000 agents due to absolute bandwidth-per-watt efficiency.

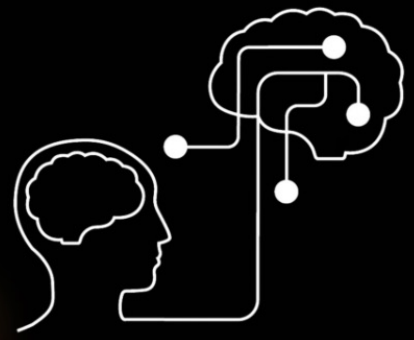
Gemini Notebook

Table 4: Racks, total facility power (PUE 1.15), and white-space footprint (racks × 30 sq ft, +45% for plant, CDU, switchgear and network). Power inherits the uncertainty in Table 7.

	100	**300**	**1000**	**5000**
Racks				
A0. Intel + SambaNova, full KV	4	6	27	129
A0c. … SnapStream, bandwidth-derived	**4**	**4**	10	40
A0m. … SnapStream, measured-anchored	5	7	19	85
A. Intel + SambaNova + NVIDIA	5	5	10	41
B. NVIDIA-native	4	4	**5**	**10**
B′. NVIDIA + Groq 3 LPX	9	9	10	15
C. Trainium3 + Cerebras	5	5	7	22
Facility power, kW				
A0. full KV	**75**	98	345	1,633
A0c. bandwidth-derived	**75**	**75**	**150**	**610**
A0m. measured-anchored	86	109	253	1,127
A	309	309	372	1,051
B	604	604	621	932
B′	1,524	1,524	1,541	1,852
C	394	394	509	1,495
White space, sq ft				
A0. full KV	174	261	1,174	5,612
A0c. bandwidth-derived	174	174	435	1,740
A0m. measured-anchored	218	304	826	3,698
A	218	218	435	1,784
B	**174**	**174**	**218**	**435**
B′	392	392	435	652
C	218	218	304	957

AI-human collaborations in cancer

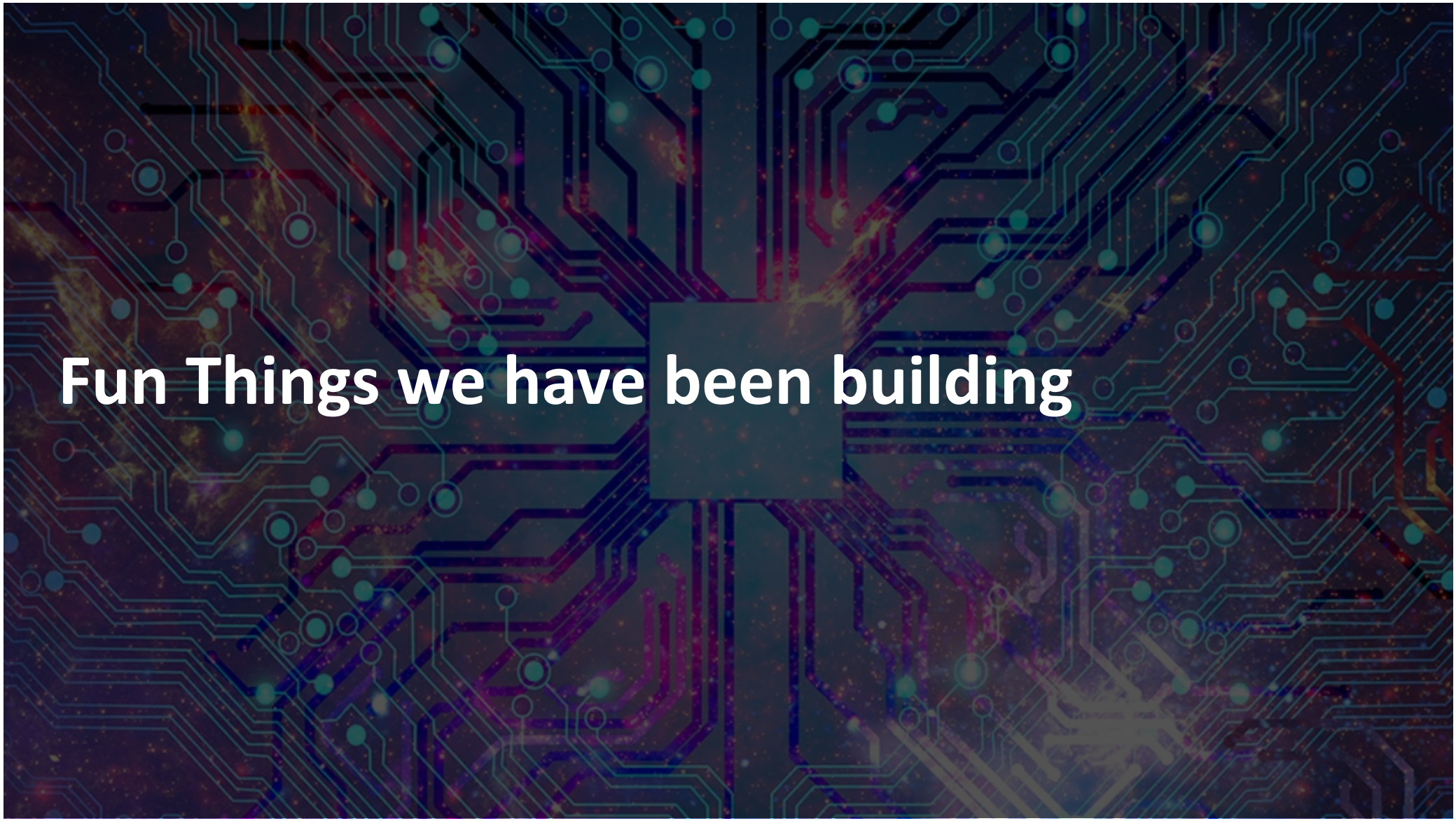
These challenges are now closed for Expressions of Interest from global, interdisciplinary research teams. Submissions were due by 23:59 BST on 18 June 2025, with awards of up to £20m ($25m) available. Shortlisted teams will be notified in July.



Challenge: Develop interdisciplinary AI agents that can generate novel cancer research hypotheses and design research plans for them to be experimentally validated.

This is one of seven new challenges.

Want to take on this challenge? We are now calling on the global research community to assemble teams and submit an Expression of Interest.


Fun Things we have been building

LUCID Extract-o-matic/Graph-o-matic

EOM (Extract-O-Matic)

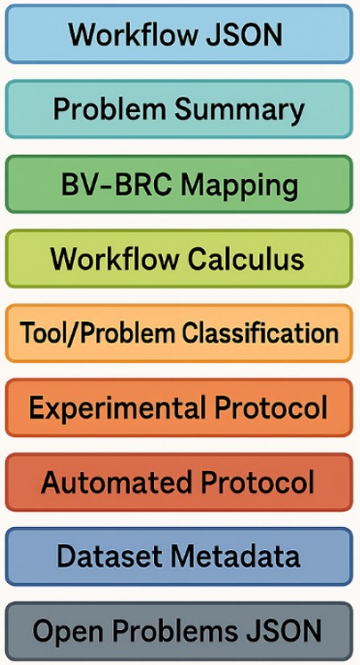
Comprehensive Bioinformatics Text Mining and Analysis Toolkit

A powerful suite of AI-powered tools for extracting, analyzing, and processing scientific information from bioinformatics literature. This toolkit combines multiple extraction modes with advanced data analysis capabilities.

🚀 Features

Core Extraction Capabilities

- **9 Extraction Modes:**
  - Workflow extraction and BV-BRC service mapping
  - Scientific problem summarization
  - Experimental protocol identification
  - Dataset extraction with comprehensive metadata
  - Hypothesis extraction with Wisteria JSON compatibility
  - Document classification (TOOL vs PROBLEM)
  - Workflow calculus notation generation
  - Automated protocol mapping for robotics/AGI

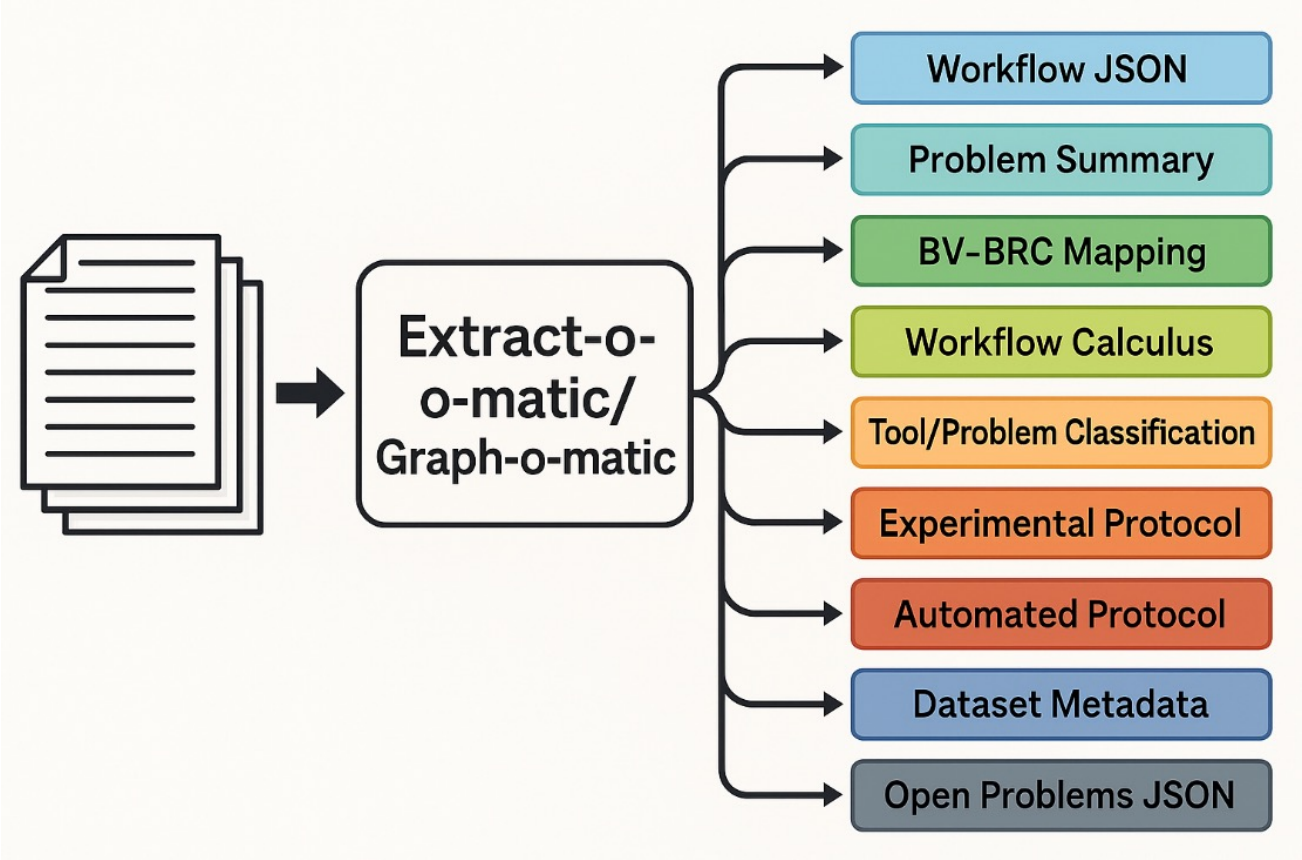


📦 Toolkit Components

Tool	Purpose	Key Features
`extract-o-matic.py`	Main extraction engine	9 modes, parallel processing, rich UI
`extract_summary_tables.py`	CSV generation from extractions	Structured data export
`analyze_datasets.py`	LLM-powered dataset analysis	Domain classification, trend analysis
`md2pdf.py`	Markdown to PDF conversion	Batch processing, report generation
`run_pipeline.py`	Complete workflow automation	End-to-end processing
`run_pipeline.sh`	Shell wrapper script	Simple execution interface

https://github.com/rick-stevens-ai/EOM

Papers to Structured Data for Computation

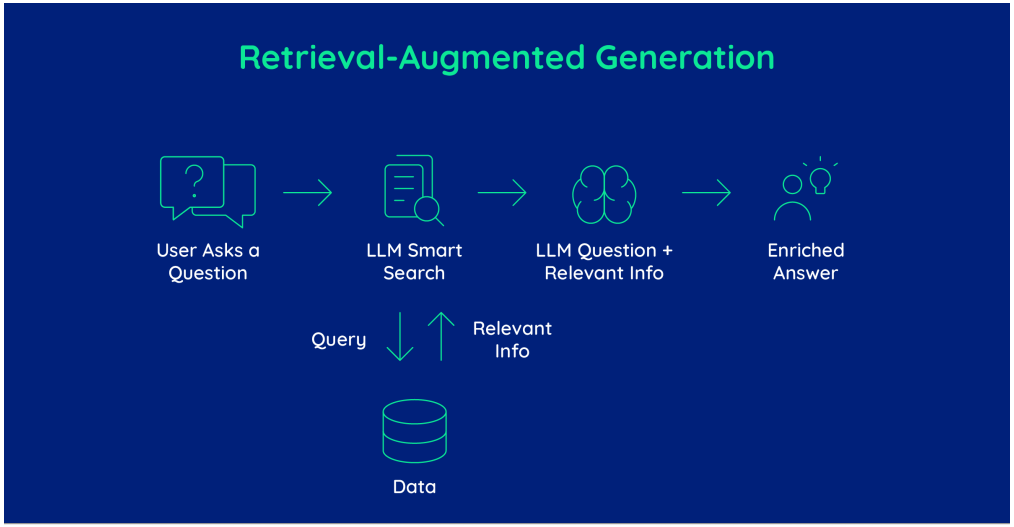


https://github.com/rick-stevens-ai/EOM

LUCID RAG System

- Improve LLM performance on LD domain related problems
- LUCID Curated database of ~200K “highly relevant” semantically parsed “chunks”
- Extracted from 14K HR papers
- Boosts performance on 6.4%-12% on LUCID benchmarks

`HR-1kp-cc-1kqa.json (7.8M)`



Student/Encoder	Base(No-Rag)	PubmedBert	SFR-Mistral	ModernBert
Llama3.3-70B-I	84.8	92.1 (+8.61%)	**95 (+12.03%)**	83.7 (-1.30%)
OpenAI-GPT4.1	89.5	93.7 (+4.69%)	**95.4 (6.59%)**	88.7 (-0.89%)
OpenAI-o3	89.2	93.9 (+5.27%)	**94.9 (6.39%)**	90.1 (+1.01%)
OpenAI-GPT3.5	0.80	86.8 (+7.43%)	**88.9 (10.02%)**	78.1 (-3.34%)
Claude4-Sonnet	87.7	92.1(+5.02%)	**94.6 (+7.87%)**	86.3 (-1.60%)
Claude4-Opus	84.8	90.2 (+6.37%)	**93.4 (+10.14%)**	75.3 (-11.20%)
Gemini2.5-Pro	TBD	TBD	TBD	TBD

Note: Values in cells are percent accuracy. Values in parentheses next to them are percent improvement (or degradation) with respect to the base (no-rag) performance of the same model.